



Snitch: Rats in the Grass - Development Handoff Document



Project Overview

Game Title: Snitch: Rats in the Grass

Type: Multiplayer Online Board Game

Current State: Functional HTML/CSS/JS Prototype

Target: Full-stack Node.js Application with Real-time Multiplayer



Core Concept

A family-friendly board game where players (as rats) travel through a snake's digestive system, making moral decisions about when to "snitch" or help others. The game teaches kids the difference between snitching (harmful tattling) and standing up for yourself or others.

Key Themes:

- Moral decision-making
- Trust vs. Self-interest
- Family-friendly humor (rats getting "pooped out")



Game Mechanics

Current Features (Prototype)

- 2-8 players
- Dice-rolling movement
- 50 space board with snake pathway
- Decision cards (moral choices)
- Random events
- Points and Trust system
- Intro animation (rats getting eaten)

- Win animation (winner gets "pooped out")
- Turn-based gameplay

Required for Production

- **Online Multiplayer** - Players join via unique link
- **Room System** - Create/join game rooms
- **Real-time Updates** - WebSocket or [Socket.io](#)
- **Persistent Game State** - Database storage
- **User Accounts** - Optional (or guest mode)

Technical Architecture

Recommended Stack

text

Frontend: React + Tailwind CSS (or Vue.js)

Backend: Node.js + Express

Real-time: Socket.io

Database: MongoDB (or PostgreSQL)

Hosting: Vercel/Netlify (frontend) + Heroku/Railway (backend)



Folder Structure

text

```
snitch-game/
├─ client/                                # React frontend
│   └─ public/
│   └─ src/
│       └─ components/
│           └─ Board.jsx
│           └─ PlayerCard.jsx
│           └─ DecisionModal.jsx
│           └─ IntroAnimation.jsx
│           └─ PoopAnimation.jsx
```

```
| | | └─ Lobby.jsx
| | └─ hooks/
| | | └─ useGameState.js
| | └─ utils/
| | | └─ gameLogic.js
| | └─ App.jsx
| └─ package.json
└─ server/                                # Node.js backend
  └─ models/
    └─ Game.js
    └─ Player.js
  └─ routes/
    └─ api.js
  └─ sockets/
    └─ gameSocket.js
  └─ utils/
    └─ gameEngine.js
  └─ server.js
  └─ package.json
└─ README.md
```



Development Tasks

Phase 1: Backend Setup (Week 1)

1.1 Initialize Node.js Project

```
bash
```

```
mkdir snitch-server
cd snitch-server
npm init -y
npm install express socket.io cors dotenv
npm install -D nodemon
```



1.2 Create Basic Server

```
javascript
```

```
// server.js
const express = require('express');
const http = require('http');
const socketIo = require('socket.io');
const cors = require('cors');

const app = express();
const server = http.createServer(app);
const io = socketIo(server, {
  cors: {
    origin: "http://localhost:3000",
    methods: ["GET", "POST"]
  }
});

app.use(cors());
app.use(express.json());

// Game rooms storage
const rooms = {};

io.on('connection', (socket) => {
  console.log('New client connected:', socket.id);

  socket.on('create-room', (data) => {
    // Create game room logic
  });

  socket.on('join-room', (data) => {
    // Join game room logic
  });

  socket.on('game-action', (data) => {
    // Handle game actions (roll dice, make decision)
  });

  socket.on('disconnect', () => {
    console.log('Client disconnected:', socket.id);
  });
});

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
```

```
    console.log(`Server running on port ${PORT}`);  
  });
```



1.3 Game State Management

javascript

```
// utils/gameEngine.js  
class GameEngine {  
  constructor(roomId, playerCount) {  
    this.roomId = roomId;  
    this.players = [];  
    this.boardSpaces = [];  
    this.currentPlayerIndex = 0;  
    this.gameOver = false;  
    this.awaitingDecision = false;  
    this.isPooping = false;  
  }  
  
  // Copy all game logic from prototype  
  generateBoard() { /* ... */ }  
  createPlayers(count) { /* ... */ }  
  handleRoll(playerId) { /* ... */ }  
  handleDecision(playerId, optionIndex) { /* ... */ }  
  // ... etc  
}
```



Phase 2: Frontend Migration (Week 2)

2.1 React Setup

bash

```
npx create-react-app snitch-client  
cd snitch-client  
npm install socket.io-client  
npm install -D tailwindcss
```



2.2 Create Core Components

- **Lobby.jsx** - Player joins/creates room
- **Board.jsx** - The game board
- **PlayerCard.jsx** - Individual player stats
- **DecisionModal.jsx** - Decision cards
- **IntroAnimation.jsx** - Eaten by snake
- **PoopAnimation.jsx** - Winner animation

2.3 Socket Integration

javascript

```
// hooks/useGameSocket.js
import { useEffect, useState } from 'react';
import io from 'socket.io-client';

const SOCKET_URL = process.env.REACT_APP_SOCKET_URL || 'http://localhost:5000';

export const useGameSocket = (roomId, playerId) => {
  const [socket, setSocket] = useState(null);
  const [gameState, setGameState] = useState(null);

  useEffect(() => {
    const newSocket = io(SOCKET_URL);
    setSocket(newSocket);

    newSocket.emit('join-room', { roomId, playerId });

    newSocket.on('game-state-update', (state) => {
      setGameState(state);
    });

    return () => newSocket.close();
  }, [roomId, playerId]);

  const sendAction = (action, data) => {
    if (socket) {
      socket.emit(action, { roomId, playerId, ...data });
    }
  };
};
```

```
    return { gameState, sendAction };  
  }  
};
```



Phase 3: Visual Improvements (Week 3)

3.1 UI/UX Design

- **Professional Color Palette:** Dark theme with vibrant accents
- **Responsive Design:** Mobile-first approach
- **Animations:** Smooth transitions with Framer Motion
- **Snake Design:** Enhanced SVG with gradient scales
- **Rat Avatars:** Customizable with accessories
- **Board:** Dynamic snake body with glowing effects

3.2 Enhanced Decision System

Current: 4 decision scenarios

Target: 30+ unique scenarios

Categories:

- School scenarios (bullying, cheating, sharing)
- Home scenarios (chores, honesty, responsibility)
- Community scenarios (helping strangers, reporting danger)
- Friendship scenarios (loyalty, gossip, trust)

Expanded Decision Structure:

javascript

```
const DECISION_SCENARIOS = {  
  school: [  
    {  
      id: 'school_1',  
      title: '📝 Test Cheating',  
      description: 'You see a classmate looking at your test answers...',  
      context: 'School',  
      ageGroup: '8-12',  
      options: [  
        {
```

```

        text: '🛡️ Tell the teacher (Gain 3 Trust)',
        points: 0,
        trust: 3,
        lesson: 'It\'s important to be honest and fair to everyone.'
    },
    {
        text: '😬 Look away (Gain 1 Trust)',
        points: 0,
        trust: 1,
        lesson: 'Sometimes it\'s best to mind your own business.'
    },
    {
        text: '📝 Let them copy (Gain 5 pts, -2 Trust)',
        points: 5,
        trust: -2,
        lesson: 'Helping someone cheat hurts everyone in the end.'
    }
]
// ... 29 more scenarios
];
};

```



3.3 Visual Design Mockups

Lobby Screen:

- Animated snake background
- Player count selector with rat avatars
- "Create Game" and "Join Game" buttons
- Room code display

Game Board:

- 3D-ish snake perspective
- Glowing path with animated scales
- Player tokens as rats with name labels
- Decision spaces with pulsing icons
- Trust meter and score display

Decision Cards:

- Full-screen modal with illustration
- Character dialogue for context
- Options as clickable cards with consequences
- Educational lesson popup after choice

Phase 4: Deployment (Week 4)

4.1 Environment Setup

```
bash
```

```
# Frontend .env
```

```
REACT_APP_API_URL=https://api.snitch-game.com
```

```
REACT_APP_SOCKET_URL=https://socket.snitch-game.com
```

```
# Backend .env
```

```
PORT=5000
```

```
MONGODB_URI=mongodb://localhost:27017/snitch
```

```
JWT_SECRET=your-secret-key
```

```
NODE_ENV=production
```



4.2 Deployment Platforms

- **Frontend:** Vercel or Netlify
- **Backend:** Railway, Heroku, or AWS
- **Database:** MongoDB Atlas or Supabase
- **WebSocket:** [Socket.io](#) with Redis for scaling

4.3 CI/CD Pipeline

```
yaml
```

```
# .github/workflows/deploy.yml
```

```
name: Deploy
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
jobs:
```

```
  deploy-frontend:
```

```
runs-on: ubuntu-latest
```

```
steps:
```

- uses: actions/checkout@v2
- name: Deploy to Vercel

```
run: |
```

```
npm install -g vercel
```

```
vercel --prod --token ${ secrets.VERCEL_TOKEN }
```

```
deploy-backend:
```

```
runs-on: ubuntu-latest
```

```
steps:
```

- uses: actions/checkout@v2
- name: Deploy to Railway

```
run: |
```

```
npm install -g railway
```

```
railway up --service=snitch-backend
```



Visual Design Requirements

Color Palette

CSS

```
:root {  
  --primary-dark: #0f0e17;  
  --primary-medium: #1a1a2e;  
  --primary-light: #16213e;  
  
  --accent-gold: #ffd93d;  
  --accent-green: #00b894;  
  --accent-purple: #6c5ce7;  
  --accent-pink: #fd79a8;  
  --accent-red: #e94560;  
  
  --snake-green-1: #1a472a;  
  --snake-green-2: #2d6a4f;  
  --snake-green-3: #40916c;
```

```
--snake-green-4: #52b788;  
}
```



Typography

- **Headings:** 'Fredoka One', cursive
- **Body:** 'Nunito', sans-serif
- **UI Elements:** 'Segoe UI', Tahoma

Illustrations

- Snake character with personality
- Rat characters with expressions
- Background scenes (sewer, park, school)
- Decision cards with relevant imagery



Expanded Decision Scenarios (30+)

School Life (8 scenarios)

1. Cheating on a test
2. Bullying in the hallway
3. Sharing lunch with a friend
4. Lying about homework
5. Standing up for a classmate
6. Reporting vandalism
7. Helping a lost student
8. Telling on a rule-breaker

Home & Family (7 scenarios)

1. Taking responsibility for a mess
2. Telling the truth about a broken item
3. Helping with chores without being asked

4. Protecting a sibling from blame
5. Sharing toys or games
6. Being honest about grades
7. Standing up to a family member

Community (8 scenarios)

1. Finding a lost wallet
2. Witnessing a shoplifting
3. Helping an elderly person
4. Reporting a safety hazard
5. Standing up to a bully
6. Helping a lost child
7. Being honest about a mistake
8. Calling for help in an emergency

Friendship (7 scenarios)

1. Gossiping about a friend
2. Keeping a friend's secret
3. Standing up for a friend
4. Being honest about a friend's behavior
5. Sharing credit for a project
6. Apologizing for a mistake
7. Including someone who's left out



API Endpoints

REST API

text

GET	/api/rooms	- List active rooms
POST	/api/rooms	- Create new room
GET	/api/rooms/:id	- Get room details
POST	/api/rooms/:id/join	- Join room

```
POST    /api/rooms/:id/start    - Start game
GET     /api/rooms/:id/state    - Get game state
```



WebSocket Events

javascript

```
// Client -> Server
'create-room'      - { playerName, playerCount }
'join-room'        - { roomId, playerName }
'start-game'       - { roomId }
'roll-dice'        - { roomId }
'make-decision'    - { roomId, optionIndex }
'leave-room'       - { roomId }
```

```
// Server -> Client
'room-created'     - { roomId, players }
'player-joined'    - { players }
'game-started'     - { gameState }
'game-update'      - { gameState }
'decision-required' - { scenario }
'game-over'        - { winner }
'error'            - { message }
```



Required NPM Packages

Backend

json

```
{
  "dependencies": {
    "express": "^4.18.2",
    "socket.io": "^4.7.2",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "mongoose": "^7.4.3",
```

```

    "bcryptjs": "^2.4.3",
    "jsonwebtoken": "^9.0.2",
    "uuid": "^9.0.0",
    "helmet": "^7.0.0",
    "compression": "^1.7.4"
  },
  "devDependencies": {
    "nodemon": "^3.0.1",
    "jest": "^29.6.2"
  }
}

```



Frontend

json

```

{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "socket.io-client": "^4.7.2",
    "react-router-dom": "^6.14.2",
    "framer-motion": "^10.14.0",
    "axios": "^1.5.0",
    "@emotion/react": "^11.11.1",
    "@emotion/styled": "^11.11.0"
  },
  "devDependencies": {
    "tailwindcss": "^3.3.3",
    "@tailwindcss/forms": "^0.5.4",
    "@tailwindcss/typography": "^0.5.9"
  }
}

```



Deployment Checklist

Pre-Launch

- ☐ SSL certificates configured
- ☐ Environment variables set
- ☐ Database indexes optimized
- ☐ WebSocket connection handling
- ☐ Error logging implemented
- ☐ Rate limiting enabled
- ☐ Security headers configured
- ☐ CORS properly set up
- ☐ Room cleanup (orphaned rooms)
- ☐ Player reconnection handling

Testing

- ☐ Unit tests for game logic
- ☐ Integration tests for API
- ☐ Load testing for WebSocket
- ☐ Cross-browser testing
- ☐ Mobile responsiveness
- ☐ Accessibility (WCAG 2.1)
- ☐ User acceptance testing

Launch

- ☐ Deploy to staging environment
- ☐ Run security audit
- ☐ Performance benchmarking
- ☐ Documentation updated
- ☐ Monitoring tools configured
- ☐ Backup strategy implemented
- ☐ Analytics integrated



Additional Feature Requests

Social Features

- Player profiles with stats
- Leaderboards
- Friend system
- Game history/replay
- Chat functionality

Monetization Options

- Cosmetic items (rat accessories, board themes)
- Expansion packs (new scenarios, boards)
- Premium features (custom games, no ads)

Accessibility

- Screen reader support
- Color blind modes
- Text size adjustment
- High contrast mode
- Keyboard navigation

Internationalization

- Multiple language support
- Culturally appropriate scenarios
- Translated UI text

July

17

Timeline Estimate

Phase	Tasks	Duration
1	Backend Setup & Core Logic	1 week
2	Frontend Migration & Socket Integration	1.5 weeks
3	Visual Improvements & Expanded Content	1.5 weeks

Phase	Tasks	Duration
4	Testing, Deployment, Polish	1 week
Total	Full MVP	5 weeks

Team Roles

- **Full-stack Developer (1-2)** - Backend API, WebSocket, Database
- **Frontend Developer (1-2)** - React, UI/UX, Animations
- **UI/UX Designer (1)** - Visual design, Wireframes, Illustrations
- **Game Designer (1)** - Scenarios, Balance, Gameplay
- **QA Engineer (1)** - Testing, Bug tracking, User testing

Resources

Documentation

- [Socket.io Docs](#)
- [React Docs](#)
- [MongoDB Docs](#)
- [Tailwind CSS Docs](#)

Inspiration

- [Board Game Geek](#)
- [Family Game Design](#)
- [Educational Games Research](#)

Tools

- [Figma](#) - UI Design
- [Miro](#) - Planning & Brainstorming
- [Postman](#) - API Testing

- [Vercel](#) - Frontend Hosting
- [Railway](#) - Backend Hosting

Success Metrics

- **Engagement:** 15-20 minute average game session
- **Retention:** 40%+ players return for second game
- **Virality:** 2.0+ invites per player
- **Learning:** 85%+ players report learning something
- **Family Rating:** 4.5/5 stars from families